

Core Language Working Group "ready" Issues for the February, 2023 meeting

References in this document reflect the section and paragraph numbering of document [WG21 N4928](#).

2072. Default argument instantiation for member functions of templates

Section: 13.9.2 [[temp.inst](#)] **Status:** ready **Submitter:** Maxim Kartashev **Date:** 2015-01-19

Default argument instantiation is described in 13.9.2 [[temp.inst](#)] paragraph 12, and although instantiation of default arguments for member functions of class templates is mentioned elsewhere a number of times, this paragraph only describes default argument instantiation for function templates.

Proposed resolution (approved by CWG 2023-02-06):

Change in 13.9.2 [[temp.inst](#)] paragraph 12 as follows:

If a **templated** function **template** `f` is called in a way that requires a default argument to be used, the dependent names are looked up, the semantics constraints are checked, and the instantiation of any template used in the default argument is done as if the default argument had been an initializer used in a function template specialization with the same scope, the same template parameters and the same access as that of the function template `f` used at that point, except that the scope in which a closure type is declared (7.5.5.2 [[expr.prim.lambda.closure](#)]) --- and therefore its associated namespaces --- remain as determined from the context of the definition for the default argument. This analysis is called default argument instantiation. The instantiated default argument is then used as the argument of `f`.

2475. Object declarations of type `cv void`

Section: 6.8.2 [[basic.fundamental](#)] **Status:** ready **Submitter:** Krystian Stasiowski **Date:** 2020-04-22

(From editorial issue [3953](#).)

Although an object cannot be defined with a type of `cv void`, there is nothing preventing a non-defining declaration of an object with that type. Should it be disallowed?

Notes from the December, 2020 teleconference:

Such declarations are permitted in C, so this question was referred to the C liaison for investigation.

CWG 2022-11-11

CWG resolved to making such declarations ill-formed.

Proposed resolution (approved by CWG 2022-12-02; amended 2023-02-06):

Change in 9.1 [[dcl.pre](#)] paragraph 7 as follows:

If the *decl-specifier-seq* contains the typedef specifier, the declaration is called a *typedef declaration* and each *declarator-id* is declared to be a *typedef-name*, synonymous with its associated type (9.2.4 [dcl.typedef]). [Note 4: Such a *declarator-id* is an identifier (11.4.8.3 [class.conv.fct]). —end note] If the *decl-specifier-seq* contains no typedef specifier, **Otherwise, if the type associated with a *declarator-id* is a function type (9.3.4.6 [dcl.fct]),** the declaration is called a *function declaration* if the type associated with a *declarator-id* is a function type (9.3.4.6 [dcl.fct]) and **. Otherwise, if the type associated with a *declarator-id* is an object or reference type, the declaration is an object declaration otherwise. Otherwise, the program is ill-formed.**

[Example:

```
int f(), x;           // OK, function declaration for f and object declaration for x
extern void g(),      // OK, function declaration for g
    y;               // error: void is not an object type

-- end example ]
```

2478. Properties of explicit specializations of implicitly-instantiated class templates

Section: 13.9.4 [temp.expl.spec] **Status:** ready **Submitter:** Mark Hall **Date:** 2021-02-02

According to 13.9.4 [temp.expl.spec] paragraph 16,

A member or a member template of a class template may be explicitly specialized for a given implicit instantiation of the class template, even if the member or member template is defined in the class template definition. An explicit specialization of a member or member template is specified using the syntax for explicit specialization.

The relationship between this construct and paragraph 14 is not clear:

Whether an explicit specialization of a function or variable template is inline, constexpr, or an immediate function is determined by the explicit specialization and is independent of those properties of the template.

(See also 9.2.6 [dcl.constexpr] paragraph 1, note 1.) Is this intended to apply to explicit specializations of members of implicitly-instantiated class templates? For example:

```
template<typename T> struct S {
    int f();
    constexpr int g();
};
template<> constexpr int S<int>::f() { // OK, constexpr?
    return 0;
}
template<> int S<int>::g() {           // OK, not constexpr?
    return 0;
}
```

There is implementation divergence on the treatment of this example. This divergence may relate to interpretation of the requirement in 9.2.6 [dcl.constexpr] paragraph 1,

If any declaration of a function or function template has a constexpr or consteval specifier, then all its declarations shall contain the same specifier.

Is an explicit specialization of a member of an implicitly-instantiated class template a declaration of that member? A similar question also applies to the constexpr specifier as specified in 9.2.7 [dcl.constinit] paragraph 1:

If the specifier is applied to any declaration of a variable, it shall be applied to the initializing declaration.

(Note that constexpr is not mentioned in 13.9.4 [temp.expl.spec] paragraph 14.) For example:

```

template<typename T> struct S {
    static constinit T x;
};
template<> int S<int>::x = 10;    // constinit required?
extern char c;
template<> short S<char>::x = c; // error, c not constant?

```

(Possibly relevant is the fact that default arguments are prohibited in explicit specializations of member functions of implicitly-instantiated class templates, per 13.9.4 [temp.expl.spec] bullet 21.3.)

CWG 2022-11-10

A specialization of a member of a class template redeclares the member of the primary template and thus the redeclaration rules apply. In passing, it was noticed that the rule governing explicit specializations in general omitted treatment of `constinit`, which was considered an oversight.

Proposed resolution (approved 2023-02-09):

Change in 13.9.4 [temp.expl.spec] paragraph 13 as follows:

Whether an explicit specialization of a function or variable template is inline, `constexpr`, **`constinit`**, or **`constexpr`** **an immediate function** is determined by the explicit specialization and is independent of those properties of the template.

2483. Language linkage of static member functions

Section: 9.11 [dcl.link] **Status:** ready **Submitter:** Davis Herring **Date:** 2021-03-11

According to 9.11 [dcl.link] paragraph 5,

A C language linkage is ignored in determining the language linkage of class members, friend functions with a trailing *requires-clause*, and the function type of class member functions.

It doesn't make sense that static member functions should behave like non-static member functions in this regard:

```

extern "C" {
    struct A {
        static void f();
        constexpr static void (*p)()=f; // error: must point to a function whose type has C language linkage
    };
}

```

Suggested resolution:

Change 9.11 [dcl.link] paragraph 5 as follows:

A C language linkage is ignored in determining the language linkage of class members, friend functions with a trailing *requires-clause*, and the function type of **`non-static`** class member functions.

Notes from the August, 2021 teleconference:

There was some question as to whether a linkage specification should affect the language linkage of any function declarators within class scope. The question was also raised as to whether some non-typedef syntax should be available for affecting language linkage, which would be a question for EWG.

Proposed resolution (approved by CWG 2022-11-10):

Change 9.11 [dcl.link] paragraph 5 as follows:

A C language linkage is ignored in determining the language linkage of class members, friend functions with a trailing *requires-clause*, and the function type of **non-static** class member functions.

2489. Storage provided by array of char

Section: 6.7.2 [[intro.object](#)] **Status:** ready **Submitter:** Jiang An **Date:** 2021-04-15

According to 6.7.2 [[intro.object](#)] paragraph 3,

If a complete object is created (7.6.2.8 [[expr.new](#)]) in storage associated with another object *e* of type “array of *N* unsigned char” or of type “array of *N* std::byte” (17.2.1 [[cstddef.syn](#)]), that array *provides storage* for the created object if...

However, note 4 in paragraph 13 indicates that a char array can also provide storage:

An operation that begins the lifetime of an array of char, unsigned char, or std::byte implicitly creates objects within the region of storage occupied by the array.

[Note 4: The array object provides storage for these objects. —end note]

The normative text and the note should be reconciled.

Proposed resolution (approved by CWG 2023-02-09):

Change in 6.7.2 [[intro.object](#)] paragraph 13 as follows:

An operation that begins the lifetime of an array of ~~char~~, unsigned char, or std::byte implicitly creates objects within the region of storage occupied by the array.

2516. Locus of *enum-specifier* or *opaque-enum-declaration*

Section: 6.4.2 [[basic.scope.pdecl](#)] **Status:** ready **Submitter:** Jiang An **Date:** 2021-10-03

According to 6.4.2 [[basic.scope.pdecl](#)] paragraph 3,

The locus of an *enum-specifier* or *opaque-enum-declaration* is immediately after the identifier (if any) in it (9.7.1 [[dcl.enum](#)]).

Equivalent wording has been present for a very long time; see, for instance, issue 1482. However, most or all implementations reject the example from that issue:

```
template<typename T> struct S { typedef char I; };  
enum E: S<E>::I { e };    // Implementations say E is undeclared in S<E>
```

In addition to recognizing current implementation practice, it would be practically useful if the locus were specified instead as after the *enum-head* or complete *opaque-enum-declaration*, as it would allow use of SFINAE in std::is_scoped_enum to distinguish between scoped and unscoped enumerations rather than requiring special compiler support.

CWG 2022-11-11

Move the locus to immediately after the *enum-head*.

Proposed resolution (approved by CWG 2023-02-06):

The locus of an *enum-specifier* or *opaque-enum-declaration* is immediately after the *identifier* (if any) in it **its** *enum-head*; the locus of an *opaque-enum-declaration* is immediately after it (9.7.1 [dcl.enum]).

2517. Useless restriction on use of parameter in *constraint-expression*

Section: 7.5.7.5 [expr.prim.req.nested] **Status:** ready **Submitter:** Richard Smith **Date:** 2019-06-10

According to 7.5.7.5 [expr.prim.req.nested] paragraph 2,

A local parameter shall only appear as an unevaluated operand (7.2.3 [expr.context]) within the *constraint-expression*. [Example 2:

```
template<typename T> concept C = requires (T a) {
    requires sizeof(a) == 4; // OK
    requires a == 0; // error: evaluation of a constraint variable
};
```

—end example]

However, a can't be used in a constant expression in any event, so the restriction is meaningless, except for ruling out an expression like `true ? true : a`, but there seems no reason to have a special rule for such a case.

Proposed resolution (approved by CWG 2023-01-06):

Remove 7.5.7.5 [expr.prim.req.nested] paragraph 2, including its example:

~~A local parameter shall only appear as an unevaluated operand (7.2.3 [expr.context]) within the *constraint-expression*. [Example 2:~~

```
template<typename T> concept C = requires (T a) {
    requires sizeof(a) == 4; // OK
    requires a == 0; // error: evaluation of a constraint variable
};
```

2518. Conformance requirements and `#error`/`#warning`

Section: 4.1.1 [intro.compliance.general] **Status:** ready **Submitter:** CWG **Date:** 2022-01-13

According to 4.1.1 [intro.compliance.general] bullet 2.2,

a conforming implementation shall issue at least one diagnostic message

for an ill-formed program that is not “no diagnostic required.” The relationship between this diagnostic message and the ones required by the `#error` and `#warning` preprocessor directives is not clear. For example, is an implementation required to emit multiple diagnostics if multiple `#error` directives are encountered, even though conformance requires only one? Could an implementation count the diagnostic emitted by `#warning` as satisfying the requirement for an ill-formed program?

See also issue 745.

Suggested resolution [SUPERSEDED]:

Add a new paragraph after 4.1.1 [intro.compliance.general] paragraph 2 as follows:

Although this document states only requirements on C++ implementations, those requirements are often easier to understand if they are phrased as requirements on programs, parts of programs, or execution of programs. Such

requirements have the following meaning:

- If a program contains no violations of the rules in Clause Clause 5 [\[lex\]](#) through Clause Clause 33 [\[thread\]](#) and Annex D, a conforming implementation shall, within its resource limits as described in Annex B, accept and correctly execute [*Footnote: ...*] that program.
- If a program contains a violation of any diagnosable rule or an occurrence of a construct described in this document as “conditionally-supported” when the implementation does not support that construct, a conforming implementation shall issue at least one diagnostic message.
- If a program contains a violation of a rule for which no diagnostic is required, this document places no requirement on implementations with respect to that program.

[Note: ... — end note]

Furthermore, a conforming implementation

- shall not accept a translation unit containing a `#error` preprocessing directive (15.8 [\[cpp.error\]](#)) unless it is part of a group skipped by conditional inclusion (15.2 [\[cpp.cond\]](#)) and
- shall issue at least one diagnostic message for each `#warning` or `#error` preprocessing directive not following a `#error` preprocessing directive, ignoring `#warning` and `#error` preprocessing directives appearing in groups skipped by conditional inclusion.

For classes and class templates, ...

Notes from the November, 2022 meeting

EWG review solicited via [cplusplus/papers#1366](https://plusplus/papers#1366).

EWG 2023-02-06

EWG agreed with the suggested resolution, but resolved to amend it to treat `static_assert` similarly.

Proposed resolution (February, 2023) [SUPERSEDED]:

1. Change and add a new paragraph after 4.1.1 [\[intro.compliance.general\]](#) paragraph 2 as follows:

Although this document states only requirements on C++ implementations, those requirements are often easier to understand if they are phrased as requirements on programs, parts of programs, or execution of programs. Such requirements have the following meaning:

- If a program contains no violations of the rules in Clause Clause 5 [\[lex\]](#) through Clause Clause 33 [\[thread\]](#) and Annex D, a conforming implementation shall, within its resource limits as described in Annex B, accept and correctly execute [*Footnote: ...*] that program.
- If a program contains a violation of a rule for which no diagnostic is required, this document places no requirement on implementations with respect to that program.
- **Otherwise, if** ~~If~~ a program contains a violation of any diagnosable rule or an occurrence of a construct described in this document as “conditionally-supported” when the implementation does not support that construct, a conforming implementation shall issue at least one diagnostic message.
- ~~If a program contains a violation of a rule for which no diagnostic is required, this document places no requirement on implementations with respect to that program.~~ [Note: During template argument deduction and substitution, certain constructs that in other contexts require a diagnostic are treated differently; see [\[temp.deduct\]](#) — end note]

Furthermore, a conforming implementation

- shall not accept a preprocessing translation unit containing a `#error` preprocessing directive (15.8 [\[cpp.error\]](#)),
- shall issue at least one diagnostic message for each `#warning` or `#error` preprocessing directive not following a `#error` preprocessing directive in a preprocessing translation unit, and
- shall not accept a translation unit with a failed `static_assert-declaration` (9.1 [\[dcl.pre\]](#)) **outside an uninstantiated template.**

For classes and class templates, ...

2. Change in 5.1 [[lex.separate](#)] paragraph 1 as follows:

The text of the program is kept in units called source files in this document. A source file together with all the headers (16.4.2.3 [[headers](#)]) and source files included (15.3 [[cpp.include](#)]) via the preprocessing directive `#include`, less any source lines skipped by any of the conditional inclusion (15.2 [[cpp.cond](#)]) preprocessing directives, is called a **preprocessing translation unit**.

3. Change in 5.2 [[lex.phases](#)] paragraph 7 as follows:

... The resulting tokens **constitute a translation unit and** are syntactically and semantically analyzed and translated ~~as a translation unit~~.

4. Change in 9.1 [[dcl.pre](#)] paragraph 10 as follows:

In a *static_assert-declaration*, the constant-expression is contextually converted to `bool` and the converted expression shall be a constant expression (7.7 [[expr.const](#)]). If the value of the expression when so converted is true, the declaration has no effect. Otherwise, **the static_assert-declaration has failed**, the program is ill-formed, and the resulting diagnostic message (4.1 [[intro.compliance](#)]) should include the text of the string-literal, if one is supplied.

CWG 2023-02-10

CWG decided to fold P2593R1 (`static_assert(false)`) into the proposed resolution.

Proposed resolution (approved by CWG 2023-02-10):

1. Change and add a new paragraph after 4.1.1 [[intro.compliance.general](#)] paragraph 2 as follows:

Although this document states only requirements on C++ implementations, those requirements are often easier to understand if they are phrased as requirements on programs, parts of programs, or execution of programs. Such requirements have the following meaning:

- If a program contains no violations of the rules in Clause 5 [[lex](#)] through Clause 33 [[thread](#)] and Annex D, a conforming implementation shall, within its resource limits as described in Annex B, accept and correctly execute [*Footnote: ...*] that program.
- If a program contains a violation of a rule for which no diagnostic is required, this document places no requirement on implementations with respect to that program.
- **Otherwise, if** ~~if~~ a program contains a violation of any diagnosable rule or an occurrence of a construct described in this document as “conditionally-supported” when the implementation does not support that construct, a conforming implementation shall issue at least one diagnostic message.
- ~~If a program contains a violation of a rule for which no diagnostic is required, this document places no requirement on implementations with respect to that program.~~ [Note: During template argument deduction and substitution, certain constructs that in other contexts require a diagnostic are treated differently; see [[temp.deduct](#)] — *end note*]

Furthermore, a conforming implementation

- shall not accept a preprocessing translation unit containing a `#error` preprocessing directive (15.8 [[cpp.error](#)]),
- shall issue at least one diagnostic message for each `#warning` or `#error` preprocessing directive not following a `#error` preprocessing directive in a preprocessing translation unit, and
- shall not accept a translation unit with a *static_assert-declaration* that fails (9.1 [[dcl.pre](#)]).

For classes and class templates, ...

2. Change in 5.1 [[lex.separate](#)] paragraph 1 as follows:

The text of the program is kept in units called source files in this document. A source file together with all the headers (16.4.2.3 [[headers](#)]) and source files included (15.3 [[cpp.include](#)]) via the preprocessing directive `#include`, less any source lines skipped by any of the conditional inclusion (15.2 [[cpp.cond](#)]) preprocessing directives, is called a **preprocessing translation unit**.

3. Change in 5.2 [lex.phases] paragraph 7 as follows:

... The resulting tokens **constitute a translation unit and** are syntactically and semantically analyzed and translated ~~as a translation unit~~.

4. Change in 9.1 [dcl.pre] paragraph 10 as follows:

In a *static_assert-declaration*, the constant-expression is contextually converted to bool and the converted expression shall be a constant expression (7.7 [expr.const]). If the value of the expression when so converted is true **or the expression is evaluated in the context of a template definition**, the declaration has no effect. Otherwise, **the static_assert-declaration fails**, the program is ill-formed, and the resulting diagnostic message (4.1 [intro.compliance]) should include the text of the string-literal, if one is supplied. [Example:

```
static_assert(sizeof(int) == sizeof(void*), "wrong pointer size");
static_assert(sizeof(int[2])); // OK, narrowing allowed
```

```
template <class T>
void f(T t) {
    if constexpr (sizeof(T) == sizeof(int)) {
        use(t);
    } else {
        static_assert(false, "must be int-sized");
    }
}

void g(char c) {
    f(0); // OK
    f(c); // error: must be int-sized
}
```

-- end example]

5. Change in 13.8 [temp.res] paragraph 6 as follows:

- ... The program is ill-formed, no diagnostic required, if:
- no valid specialization, **ignoring static_assert-declarations that fail**, can be generated for a template or a substatement of a constexpr if statement (8.5.2 [stmt.if]) within a template and the template is not instantiated, or
 - ...

2520. Template signature and default template arguments

Section: 3.54 [defns.signature.templ] **Status:** ready **Submitter:** John Spicer **Date:** 2022-01-25

According to 3.54 [defns.signature.templ], the signature of a function template includes:

name, parameter-type-list, enclosing namespace, return type, *template-head*, and trailing *requires-clause* (if any)

The mention of the *template-head* is a change from previous versions of the Standard, which referred to the “template parameter list”, in order to include the *requires-clause* in the signature. However, *template-head* is a syntactic nonterminal and thus includes everything in that production, including default template arguments. It seems undesirable to make the default arguments part of the template signature.

CWG 2022-11-11

CWG agrees with the suggested direction.

Proposed resolution (approved by CWG 2023-02-09):

1. Change in 3.53 [[defns.signature.friend](#)] as follows:

```
signature
<function template>
name, parameter-type-list, enclosing namespace, return type, signature of the template-head, and trailing
requires-clause (if any)
```

2. Change in 3.54 [[defns.signature.templ](#)] as follows:

```
signature
<friend function template with constraint involving enclosing template parameters>
name, parameter-type-list, return type, enclosing class, signature of the template-head, and trailing
requires-clause (if any)
```

3. Change in 3.57 [[defns.signature.member](#)] as follows:

```
signature
<class member function template>
name, parameter-type-list, class of which the function is a member, cv-qualifiers (if any), ref-qualifier (if
any), return type (if any), signature of the template-head, and trailing requires-clause (if any)
```

4. Add a new section in Clause 3 [[intro.defs](#)] as follows:

```
signature
<template-head>
template parameter list, excluding template parameter names and default arguments, and requires-clause (if
any)
```

2521. User-defined literals and reserved identifiers

Section: 12.6 [[over.literal](#)] **Status:** ready **Submitter:** Jim X **Date:** 2022-01-07

The example in 12.6 [[over.literal](#)] paragraph 8 has the following lines:

```
double operator""_Bq(long double); // OK: does not use the reserved identifier _Bq (5.10)
double operator"" _Bq(long double); // ill-formed, no diagnostic required:
                                     // uses the reserved identifier _Bq (5.10)
```

The referenced rule in 5.10 [[lex.name](#)] is in bullet 3.1:

Each identifier that contains a double underscore __ or begins with an underscore followed by an uppercase letter is reserved to the implementation for any use.

The distinction being drawn in the user-defined literal example apparently relies on the grammar for *literal-operator-id* at the beginning of 12.6 [[over.literal](#)]:

```
literal-operator-id:
  operator string-literal identifier
  operator user-defined-string-literal
```

The second production does not mention the syntactic non-terminal *identifier*, so the *literal-operator-id* operator""_Bq presumably does not run afoul of the restriction in 5.10 [[lex.name](#)]. However, the grammar for *user-defined-string-literal* in 5.13.8 [[lex.ext](#)] is:

```
user-defined-string-literal:
  string-literal ud-suffix

ud-suffix:
```

identifier

There doesn't seem to be a rule that exempts the *identifier* that is the *ud-suffix* of a *user-defined-string-literal* from the restriction in 5.10 [lex.name]. Either the example is incorrect or there needs to be a refinement of the rule in 5.10 [lex.name].

CWG 2022-11-11

CWG feels that the ostensible significance of whitespace in this context is unfortunate. In addition, since the normative rule is not consistent with the example, CWG solicits EWG input on the handling of this issue via [cplusplus/papers#1367](https://ericniebler.com/papers#1367).

EWG 2023-02-06

EWG had consensus on "The form of User Defined Literals that permits a space between the quotes and the name of the literal should be deprecated, and eventually removed. Additionally, the UDL name should be excluded from the restriction in 5.10 [lex.name] in the non-deprecated form (sans space)."

Proposed resolution (February, 2023) [SUPERSEDED]:

1. Change in 5.10 [lex.name] paragraph 3 as follows:

In addition, some identifiers **appearing as a token or pp-token** are reserved for use by C++ implementations and shall not be used otherwise; no diagnostic is required.

2. Change in 12.6 [over.literal] paragraph 1 as follows:

The *string-literal* or *user-defined-string-literal* in a *literal-operator-id* shall have no *encoding-prefix* and shall contain no characters other than the implicit terminating '\0'. The *ud-suffix* of the *user-defined-string-literal* or the identifier in a *literal-operator-id* is called a *literal suffix identifier*. **The first form of literal-operator-id is deprecated.** Some literal suffix identifiers are reserved for future standardization; see 16.4.5.3.6 [usrlit.suffix]. A declaration whose *literal-operator-id* uses such a literal suffix identifier is ill-formed, no diagnostic required.

3. Change in 12.6 [over.literal] paragraph 8 as follows:

```
void operator ""_km(long double);           // OK
string operator "" _i18n(const char*, std::size_t); // OK, deprecated
template <char...> double operator "" _u03C0(); // OK, UCN for lowercase pi
float operator "" _e(const char*);           // OK
float operator "" E(const char*);            // ill-formed, no diagnostic required:
                                           // reserved literal suffix ([usrlit.suffix], [lex.ext])
double operator "" _Bq(long double);         // OK, does not use the reserved identifier _Bq ([lex.name])
double operator "" _Bq(long double);         // ill-formed, no diagnostic required:
                                           // uses the reserved identifier _Bq ([lex.name])
float operator "" _B(const char*);           // error: non-empty string-literal
string operator "" _5X(const char*, std::size_t); // error: invalid literal suffix identifier
double operator "" _miles(double);          // error: invalid parameter-declaration-clause
template <char...> int operator "" _j(const char*); // error: invalid parameter-declaration-clause
extern "C" void operator "" _m(long double); // error: C language linkage
```

4. Add a new subclause after D.8 [depr.impldec]:

D.9 Literal operator function declarations using an identifier [depr.lit]

A *literal-operator-id* (12.6 [over.literal]) of the form

```
operator string-literal identifier
```

is deprecated.

CWG 2023-02-07

Some implementers are concerned about the lack of space for implementation extensions. The suggestion is to reserve literal suffix identifiers starting with two underscores for the implementation in 16.4.5.3.6 [usrlit.suffix]. EWG is invited to comment

on that direction.

EWG / LEWG 2023-02-07

EWG and LEWG resolved to amend the proposed resolution for CWG2521 to reserve literal suffix identifiers with double underscores anywhere for implementation use.

Proposed resolution (approved by CWG 2023-02-09):

1. Change in 5.10 [\[lex.name\]](#) paragraph 3 as follows:

In addition, some identifiers **appearing as a token or pp-token** are reserved for use by C++ implementations and shall not be used otherwise; no diagnostic is required.

2. In 5.13.8 [\[lex.ext\]](#), modify all occurrences as follows:

```
operator ""_x
```

3. Change in 5.13.8 [\[lex.ext\]](#) paragraph 7 as follows:

```
long double operator ""_w(long double);
std::string operator ""_w(const char16_t*, std::size_t);
unsigned operator ""_w(const char*);
int main() {
    1.2_w;      // calls operator ""_w(1.2L)
    u"one"_w;    // calls operator ""_w(u"one", 3)
    12_w;       // calls operator ""_w("12")
    "two"_w;     // error: no applicable literal operator
}
```

4. Change in 12.6 [\[over.literal\]](#) paragraph 1 as follows:

The *string-literal* or *user-defined-string-literal* in a *literal-operator-id* shall have no *encoding-prefix* and shall contain no characters other than the implicit terminating `'\0'`. The *ud-suffix* of the *user-defined-string-literal* or the identifier in a *literal-operator-id* is called a *literal suffix identifier*. **The first form of *literal-operator-id* is deprecated.** Some literal suffix identifiers are reserved for future standardization; see 16.4.5.3.6 [\[usrlit.suffix\]](#). A declaration whose *literal-operator-id* uses such a literal suffix identifier is ill-formed, no diagnostic required.

5. Change in 12.6 [\[over.literal\]](#) paragraph 8 as follows:

```
void operator ""_km(long double);           // OK
string operator "" _i18n(const char*, std::size_t); // OK, deprecated
template <char...> double operator ""_\u03C0(); // OK, UCN for lowercase pi
float operator ""_e(const char*);             // OK
float operator ""E(const char*);              // ill-formed, no diagnostic required:
// reserved literal suffix ([usrlit.suffix], [lex.ext])
double operator ""_Bq(long double);           // OK, does not use the reserved identifier _Bq ([lex.name])
double operator "" _Bq(long double);          // ill-formed, no diagnostic required:
// uses the reserved identifier _Bq ([lex.name])
float operator ""_B(const char*);             // error: non-empty string-literal
string operator ""_5X(const char*, std::size_t); // error: invalid literal suffix identifier
double operator ""_miles(double);             // error: invalid parameter-declaration-clause
template <char...> int operator ""_j(const char*); // error: invalid parameter-declaration-clause
extern "C" void operator ""_m(long double);    // error: C language linkage
```

6. Change in 16.4.5.3.6 [\[usrlit.suffix\]](#) as follows:

Literal suffix identifiers (12.6 [\[over.literal\]](#)) that do not start with an underscore are reserved for future standardization. **Literal suffix identifiers that contain a double underscore __ are reserved for use by C++ implementations.**

7. Add a new subclause after D.8 [\[depr.impldec\]](#):

D.9 Literal operator function declarations using an identifier [depr.lit]

A *literal-operator-id* (12.6 [over.literal]) of the form

`operator string-literal identifier`

is deprecated.

2523. Undefined behavior via omitted destructor call in constant expressions

Section: 7.7 [expr.const] **Status:** ready **Submitter:** Jiang An **Date:** 2021-09-06

According to 7.7 [expr.const] bullet 5.8, one criterion that disqualifies an expression from being a core constant expression is:

an operation that would have undefined behavior as specified in Clause 4 [intro] through Clause 15 [cpp]

One potential source of undefined behavior is the omission of a call to a destructor for a constructed object, as described in 6.7.3 [basic.life] paragraph 5:

A program may end the lifetime of an object of class type without invoking the destructor, by reusing or releasing the storage as described above. [Note 3: A *delete-expression* (7.6.2.9 [expr.delete]) invokes the destructor prior to releasing the storage. —end note] In this case, the destructor is not implicitly invoked and any program that depends on the side effects produced by the destructor has undefined behavior.

For example:

```
#include <memory>

constexpr int test_basic_life_p5() {
    class guard_t {
        int &ref_;
    public:
        explicit constexpr guard_t(int &i) : ref_{i} {}
        constexpr ~guard_t() { ref_ = 42; }
    };

    int result = 0;

    auto alloc = std::allocator<guard_t>{};
    auto pguard = alloc.allocate(1);
    std::construct_at(pguard, result);
    // std::destroy_at(pguard);
    alloc.deallocate(pguard, 1);

    return result; // value depends on destructor execution
}

int main() {
    constexpr auto v = test_basic_life_p5();
    return v;
}
```

It is not clear that it is reasonable to require implementations to diagnose this form of undefined behavior in constant expressions.

A somewhat related question is raised by the restrictions on the use of `longjmp` in 17.13.3 [csetjmp.syn] paragraph 2:

A `setjmp/longjmp` call pair has undefined behavior if replacing the `setjmp` and `longjmp` by `catch` and `throw` would invoke any non-trivial destructors for any objects with automatic storage duration.

Here the undefined behavior occurs for any non-trivial destructor that is skipped, not just one for which the program depends on its side effects, as in 6.7.3 [basic.life] paragraph 5. Perhaps these two specifications should be harmonized.

Additional notes (April, 2022):

The phrase "[a] program that depends on the side effects" may have these meanings:

- the program depends on the side effects if it would have undefined behavior if they don't happen (which they don't): this would mean the second half of the sentence has no normative effect and can be struck
- the program depends on the side effects if it would have different observable behavior if (in violation of the standard) the destructor were actually invoked by the implementation when the object's lifetime ended
- something else

The second option would need a fork in the evaluation of constant expressions to determine whether undefined behavior occurs.

Proposed resolution (approved by CWG 2022-11-11):

Change in 6.7.3 [\[basic.life\]](#) paragraph 5 as follows:

A program may end the lifetime of an object of class type without invoking the destructor, by reusing or releasing the storage as described above. [Note 3: A *delete-expression* (7.6.2.9 [\[expr.delete\]](#)) invokes the destructor prior to releasing the storage. —end note] In this case, the destructor is not implicitly invoked and any program that depends on the side effects produced by the destructor has undefined behavior. [Note: The correct behavior of a program often depends on the destructor being invoked for each object of class type. -- end note]

2526. Relational comparison of void* pointers

Section: 7.6.9 [\[expr.rel\]](#) **Status:** ready **Submitter:** Paul Keir **Date:** 2020-06-15

Prior to the adoption of paper N3624 (resolving issue 1512), comparison of void* pointers was explicitly unspecified. The current wording of 7.6.9 [\[expr.rel\]](#), however, describes only comparison of “pointers to objects” (paragraphs 4 and 5), but a pointer to void is not a pointer to an object, only an object pointer type (as opposed to a function pointer type). Formally, that means that comparing void* pointers is undefined behavior, which seems undesirable.

As a related note, there is implementation divergence over whether relational comparisons of void* pointers are accepted in constant expressions (when the void* values are converted from pointers that would otherwise be comparable in constant expressions).

CWG 2022-11-11

Paper N3624 erroneously removed support for void* and function pointer relational comparisons. That ought to be restored.

Proposed resolution (approved by CWG 2023-02-09):

Change in 7.6.9 [\[expr.rel\]](#) paragraph 5 as follows:

... Otherwise, the result of each of the operators is unspecified. [Note: A relational operator applied to unequal function pointers or to unequal pointers to void yields an unspecified result. -- end note]

2528. Three-way comparison and the usual arithmetic conversions

Section: 7.4 [\[expr.arith.conv\]](#) **Status:** ready **Submitter:** Cameron DaCamara **Date:** 2022-01-26

Consider an example like:

```
void f(unsigned char i, unsigned ui) {  
    i <=> ui;  
}
```

}

According to 7.6.8 [expr.spaceship] paragraph 4, the usual arithmetic conversions are applied to the operands. According to 7.4 [expr.arith.conv] bullet 1.5, the integral promotions are performed on both operands, resulting in `i` being converted from unsigned `char` to `int`. The operands are then of types `int` and unsigned `int`, so bullet 1.5.5 applies, further converting `i` to type unsigned `int`.

Unfortunately, that latter conversion, from `int` to unsigned `int`, is a narrowing conversion, which runs afoul of 7.6.8 [expr.spaceship] bullet 4.1, which prohibits narrowing conversions other than integral to floating in three-way comparisons.

Suggested resolution [SUPERSEDED]:

Change 7.4 [expr.arith.conv] bullet 1.5 as follows:

Otherwise, the integral promotions (7.3.7 [conv.prom]) shall be performed on both operands **each operand shall be converted to a common type C. The integral promotion rules (7.3.7 [conv.prom]) shall be used to determine a type T1 and type T2 for each operand.**⁵⁰ Then the following rules shall be applied to the promoted operands **determine C:**

- If both operands have **T1 and T2 are** the same type, **no further conversion is needed C shall be that type.**
- Otherwise, if both operands have **T1 and T2 are both** signed integer types or both **have are** unsigned integer types, **the operand with the type of lesser integer conversion rank shall be converted to the type of the operand C shall be the type** with greater rank.
- Otherwise, if the operand that has **the type U that is an** unsigned integer type has rank greater than or equal to the rank of the **other** type of the other operand, **the operand with signed integer type shall be converted to the type of the operand with unsigned integer type, C shall be U.**
- Otherwise, if the type of the operand with **S that is a** signed integer type can represent all of the values of the **other** type of the operand with unsigned integer type, **the operand with unsigned integer type shall be converted to the type of the operand with signed integer type, C shall be S.**
- Otherwise, **both operands shall be converted to C shall be** the unsigned integer type corresponding to the type of the operand with signed integer type.

Proposed resolution (approved by CWG 2023-02-09):

Change in 7.4 [expr.arith.conv] bullet 1.3 as follows, adding sub-bullets:

Otherwise, the integral promotions (7.3.7 [conv.prom]) are performed on both operands **each operand is converted to a common type C. The integral promotion rules (7.3.7 [conv.prom]) are used to determine a type T1 and type T2 for each operand.** [Footnote: ...] Then the following rules are applied to the promoted operands **determine C:**

- If both operands have **T1 and T2 are** the same type, **no further conversion is needed C is that type.**
- Otherwise, if **T1 and T2 are** both operands have signed integer types or **are** both have unsigned integer types, **the operand with the type of lesser integer conversion rank is converted to the type of the operand C is the type** with greater rank.
- Otherwise, if the operand that has **let U be the** unsigned integer type **and S be the signed integer type.**
 - **If U** has rank greater than or equal to the rank of the type of the other operand, **the operand with signed integer type is converted to the type of the operand with unsigned integer type S, C is U.**
 - Otherwise, if the type of the operand with signed integer type **S** can represent all of the values of the type of the operand with unsigned integer type, **the operand with unsigned integer type is converted to the type of the operand with signed integer type U, C is S.**
 - Otherwise, **both operands are converted to C is** the unsigned integer type corresponding to the type of the operand with signed integer type **S.**

2529. Constant destruction of constexpr references

Section: 7.7 [[expr.const](#)] **Status:** ready **Submitter:** Jiang An **Date:** 2022-02-08

According to 9.2.6 [[dcl.constexpr](#)] paragraph 10, a constexpr variable must have constant destruction. However, 7.7 [[expr.const](#)] paragraph 7 only defines constant destruction for objects, not for references. Presumably constexpr references should also be able to have constant destruction, and any temporary object to which such a reference is bound should also be required to have constant destruction.

Proposed resolution (approved by CWG 2022-12-02):

Change in 9.2.6 [[dcl.constexpr](#)] paragraph 7 as follows:

A constexpr specifier used in an object declaration declares the object as const. Such an object shall have literal type and shall be initialized. In any constexpr variable declaration, the full-expression of the initialization shall be a constant expression (7.7 [[expr.const](#)]). A constexpr variable **that is an object, as well as any temporary to which a constexpr reference is bound**, shall have constant destruction. [Example: ...]

2530. Multiple definitions of enumerators

Section: 6.3 [[basic.def.odr](#)] **Status:** ready **Submitter:** Naiver Miigon **Date:** 2022-02-10

Issue 2494 specified a list of definable items and required that no translation unit contain more than one definition of any of those items. However, the list omits enumeration constants, implicitly allowing an example like:

```
enum E { e, e };
```

According to 6.1 [[basic.pre](#)] paragraph 3, an enumerator is an entity. According to 6.2 [[basic.def](#)] paragraph 2,

Each entity declared by a *declaration* is also *defined* by that declaration unless: ...

and enumerators are not on the list of excluded cases, so an *enumerator-definition* is a definition. Furthermore, 6.6 [[basic.link](#)] paragraph 8 says,

Two declarations of entities declare the same entity if, considering declarations of unnamed types to introduce their names for linkage purposes, if any (9.2.4 [[dcl.typedef](#)], 9.7.1 [[dcl.enum](#)]), they correspond (6.4.1 [[basic.scope.scope](#)]), have the same target scope that is not a function or template parameter scope, and either

- they appear in the same translation unit, or
- ...

In the example above, both enumerators thus define the same entity, so the one-definition rule is responsible for excluding the duplicate definitions but does not do so.

Suggested resolution [SUPERSEDED]:

Change 6.3 [[basic.def.odr](#)] paragraph 1 as follows:

Each of the following is termed a *definable item*:

- ...
- an enumeration type (9.7.1 [[dcl.enum](#)]),
- **an enumerator**,
- a function (9.3.4.6 [[dcl.fct](#)]),
- ...

Proposed resolution (approved by CWG 2022-12-02):

Change in 9.7.1 [[dcl.enum](#)] paragraph 2 as follows:

... The identifiers in an *enumerator-list* are declared as constants, and can appear wherever constants are required. **The same identifier shall not appear as the name of multiple enumerators in an *enumerator-list*.** An *enumerator-definition* with = gives the associated *enumerator* the value indicated by the *constant-expression*. If the first *enumerator* has no *initializer*, the value of the corresponding constant is zero. An *enumerator-definition* without an *initializer* gives the *enumerator* the value obtained by increasing the value of the previous *enumerator* by one.

2539. Three-way comparison requiring strong ordering for floating-point types

Section: 11.10.3 [[class.spaceship](#)] **Status:** ready **Submitter:** Richard Smith **Date:** 2022-02-24

Consider:

```
struct MyType {
    int i;
    double d;
    std::strong_ordering operator<=> (const MyType& c) const = default;
};
```

The defaulted three-way comparison operator is defined only if it is used, per 11.10.1 [[class.compare.default](#)] paragraph 1:

A comparison operator function for class C that is defaulted on its first declaration and is not defined as deleted is implicitly defined when it is odr-used or needed for constant evaluation.

The current rules make an odr-use of the three-way comparison operator ill-formed, but it would be preferable if it were deleted instead. In particular, 11.10.3 [[class.spaceship](#)] bullet 2.2 specifies

If the synthesized three-way comparison of type R between any objects x_i and x_j is not defined, the operator function is defined as deleted.

This refers to bullets 1.2 and 1.3 of 11.10.3 [[class.spaceship](#)] paragraph 1:

The *synthesized three-way comparison* of type R (17.11.2 [[cmp.categories](#)]) of glvalues a and b of the same type is defined as follows:

- If $a <=> b$ is usable (11.10.1 [[class.compare.default](#)]), `static_cast<R>(a <=> b)`.
- Otherwise, if overload resolution for $a <=> b$ is performed and finds at least one viable candidate, the synthesized three-way comparison is not defined.
- Otherwise, if R is not a comparison category type, or either the expression $a == b$ or the expression $a < b$ is not usable, the synthesized three-way comparison is not defined.
- Otherwise, ...

However, $a <=> b$ is actually usable, because 11.10.1 [[class.compare.default](#)] paragraph 3 defines:

A binary operator expression $a @ b$ is *usable* if either

- a or b is of class or enumeration type and overload resolution (12.2 [[over.match](#)]) as applied to $a @ b$ results in a usable candidate, or
- neither a nor b is of class or enumeration type and $a @ b$ is a valid expression.

`MyType().d <=> MyType().d` is a valid expression.

Proposed resolution (approved by CWG 2022-11-11) [SUPERSEDED]:

The *synthesized three-way comparison* of type R (17.11.2 [[cmp.categories](#)]) of glvalues a and b of the same type is defined as follows:

- If `a <=> b` is usable (11.10.1 [[class.compare.default](#)]) **and**
 - **if overload resolution for a direct-initialization of an object or reference of type R from `a <=> b` results in a usable candidate**, `static_cast<R>(a <=> b)`,
 - **otherwise, the synthesized three-way comparison is not defined.**
- Otherwise, if overload resolution for `a <=> b` is performed and finds at least one viable candidate, the synthesized three-way comparison is not defined.
- Otherwise, if R is not a comparison category type, or either the expression `a == b` or the expression `a < b` is not usable, the synthesized three-way comparison is not defined.
- Otherwise, ...

CWG 2023-02-06

A simplification of the wording is sought.

Proposed resolution (approved by CWG 2023-02-07):

The *synthesized three-way comparison* of type R (17.11.2 [[cmp.categories](#)]) of glvalues a and b of the same type is defined as follows:

- If `a <=> b` is usable (11.10.1 [[class.compare.default](#)]) **and can be explicitly converted to R using `static_cast`**, `static_cast<R>(a <=> b)`.
- Otherwise, if overload resolution for `a <=> b` is performed and finds at least one viable candidate, the synthesized three-way comparison is not defined.
- Otherwise, if R is not a comparison category type, or either the expression `a == b` or the expression `a < b` is not usable, the synthesized three-way comparison is not defined.
- Otherwise, ...

2543. `constexpr` and optimized dynamic initialization

Section: 9.2.7 [[dcl.constinit](#)] **Status:** ready **Submitter:** Zhihao Yuan **Date:** 2022-03-01

Subclause 9.2.7 [[dcl.constinit](#)] paragraph 2 states:

If a variable declared with the `constexpr` specifier has dynamic initialization (6.9.3.3 [[basic.start.dynamic](#)]), the program is ill-formed. [*Note:* The `constexpr` specifier ensures that the variable is initialized during static initialization (6.9.3.2 [[basic.start.static](#)]). —*end note*]

Subclause 6.9.3.2 [[basic.start.static](#)] paragraph 3 gives permission for an implementation to perform static initialization in lieu of dynamic initialization:

An implementation is permitted to perform the initialization of a variable with static or thread storage duration as a static initialization even if such initialization is not required to be done statically, provided that ...

`constexpr` will assuredly not give a diagnostic for variables that are constant initialized (7.7 [[expr.const](#)] paragraph 2), because those are required to be statically initialized and thus will never be dynamically initialized. Conversely, `constexpr` is guaranteed to give a diagnostic for variables that cannot be statically initialized, for example those with an initializer whose value depends on runtime conditions.

Between those boundaries, it is unclear whether `constexpr` ought to give a diagnostic for variables whose initializer does not satisfy the constraints of constant-initialized, yet the implementation takes advantage of the permission to turn dynamic initialization into static initialization. For example,

```
float f;
constexpr int * pi = (int*) &f;    // reinterpret_cast, not constant-initialized
```

The current wording seems to imply that `constexpr` accurately reflects whether dynamic initialization was turned into static initialization by the implementation. However, that is impossible to implement, because such decisions are often made by the optimizer, which runs later than the compiler front-end interpreting the program text containing `constexpr`.

There is value in permitting `constexpr` not to diagnose some of the dynamic initializations that are turned into static initializations.

There is also value in having portable semantics of `constexpr`.

See also issue 2536.

Notes from the November, 2022 meeting

CWG seeks the advice of EWG how to proceed with this issue, via [cplusplus/papers#1379](https://ericniebler.com/2022/11/14/cplusplus/papers#1379).

EWG 2023-01-19

EWG resolved to desire portable, guaranteed semantics for `constexpr`. That means, `constexpr` should produce a diagnostic if de-jure dynamic initialization is turned into de-facto static initialization as permitted by 6.9.3.2 [[basic.start.static](#)] paragraph 3.

Proposed resolution (approved by CWG 2023-02-06):

Change in 9.2.7 [[dcl.constinit](#)] paragraph 2 as follows:

If a variable declared with the `constexpr` specifier has dynamic initialization (6.9.3.3 [[basic.start.dynamic](#)]), the program is ill-formed, **even if the implementation would perform that initialization as a static initialization (6.9.3.2 [[basic.start.static](#)])**. [Note 1: The `constexpr` specifier ensures that the variable is initialized during static initialization (~~6.9.3.2 [[basic.start.static](#)]~~). —end note]

2558. Uninitialized subobjects as a result of an immediate invocation

Section: 7.7 [[expr.const](#)] **Status:** ready **Submitter:** Aaron Ballman **Date:** 2022-03-29

Consider:

```
struct A {
    int n;
    constexpr A() {}
};
constexpr A a; // implementations reject
```

Paper [P1331R2](#) (Permitting trivial default initialization in `constexpr` contexts) dropped the restriction that immediate invocations cannot yield results with some subobjects left uninitialized. It is unclear whether that change was intentional or accidental.

Furthermore, indeterminate values of pointer type are currently not permitted as the result of a constant expression per 7.7 [[expr.const](#)] bullet 12.2; indeterminate values of scalar types are permitted only due to the absence of a restriction.

This issue is closely related to issue 2536.

2022-12-03

Forwarded to EWG with [cplusplus/papers#1380](https://ericniebler.com/2022/12/03/cplusplus/papers#1380).

EWG 2023-01-19

Uninitialized non-variant direct subobjects should not be allowed to appear in the result of a constant expression. There was no consensus in support of the statements "Union types shall be initialized such that they have an active member in the result of a constant expression" and "EWG confirms that padding bits and data belonging to non-active variant members are permitted to have indeterminate values in the static initialization of objects".

Proposed resolution (approved by CWG 2023-01-27):

Change in 7.7 [expr.const] paragraph 12 as follows:

A constant expression is either a glvalue core constant expression that refers to an entity that is a permitted result of a constant expression (as defined below), or a prvalue core constant expression whose value satisfies the following constraints:

- if the value is an object of class type, each non-static data member of reference type refers to an entity that is a permitted result of a constant expression,
- **if the value is an object of scalar type, it does not have indeterminate value (6.7.4 [basic.indet]),**
- ...
- if the value is an object of class or array type, each subobject satisfies these constraints for the value.

2602. consteval defaulted functions

Section: 9.2.6 [dcl.constexpr] **Status:** ready **Submitter:** Aaron Ballman **Date:** 2022-06-16

It is not clear whether a defaulted consteval function is still an immediate function even if it is not a valid constexpr function. For example:

```
template <typename Ty>
struct A {
    Ty n;
    consteval A() {}
};

template <typename Ty>
struct B {
    Ty n;
    consteval B() = default;
};

A<int> a;
B<int> b;
```

The declarations of a and b should both fail due to an uninitialized member n in each of A and B. The = default; should not make a difference. However, there is implementation divergence. We should be able to lean on 7.7 [expr.const] bullet 5.5 to handle this when the immediate invocation is required.

Possible resolution:

Change in 9.2.6 [dcl.constexpr] paragraph 7 as follows:

If the instantiated template specialization of a constexpr **templated** function ~~template or member function of a class template~~ would fail to satisfy the requirements for a constexpr function, that specialization is still a constexpr function, even though a call to such a function cannot appear in a constant expression. **Similarly, if the instantiated template specialization of a consteval templated function would fail to satisfy the requirements for a consteval function, that specialization is still an immediate function, even though an immediate invocation would be ill-formed.** If no specialization of the template would satisfy the requirements for a constexpr **or consteval** function when considered as a non-template function, the template is ill-formed, no diagnostic required.

Proposed resolution (August, 2022) [SUPERSEDED]:

Change in 9.2.6 [dcl.constexpr] paragraph 4 as follows:

If the instantiated template specialization of a constexpr **templated** function ~~template or member function of a class template~~ would fail to satisfy the requirements for a constexpr function, that specialization is still a constexpr function, even though a call to such a function cannot appear in a constant expression. **Similarly, if the instantiated template specialization of a consteval templated function would fail to satisfy the requirements**

for a consteval function, that specialization is still an immediate function, even though an immediate invocation would be ill-formed.

Additional notes (November, 2022)

The proposed wording is possibly not addressing the point of the issue; the issue has been retracted from the WG21 plenary straw polls for further consideration in CWG.

Proposed resolution (approved by CWG 2023-01-27):

1. Change in 9.2.6 [\[dcl.constexpr\]](#) paragraph 3 as follows:

~~The definition of a constexpr function shall satisfy the following requirements:~~ **A function is constexpr-suitable if:**

- it ~~shall~~ **is** not ~~be~~ a coroutine (9.5.4 [\[dcl.fct.def.coroutine\]](#)); **, and**
- if the function is a constructor or destructor, its class ~~shall~~ **does** not have any virtual base classes.

Except for instantiated constexpr functions, non-templated constexpr functions shall be constexpr-suitable.

2. Delete 9.2.6 [\[dcl.constexpr\]](#) paragraph 4:

~~If the instantiated template specialization of a constexpr function template or member function of a class template would fail to satisfy the requirements for a constexpr function, that specialization is still a constexpr function, even though a call to such a function cannot appear in a constant expression.~~

3. Change in 7.5.5.2 [\[expr.prim.lambda.closure\]](#) paragraph 5 as follows:

... The function call operator or any given operator template specialization is a constexpr function if either the corresponding *lambda-expression's parameter-declaration-clause* is followed by constexpr or consteval, or it ~~satisfies the requirements for a constexpr function~~ **is constexpr-suitable** (9.2.6 [\[dcl.constexpr\]](#)). ...

4. Change in 7.7 [\[expr.const\]](#) bullet 5.5 as follows:

- an invocation of an instantiated constexpr function that ~~fails to satisfy the requirements for a constexpr function~~ **is not constexpr-suitable**;

5. Change in 9.5.2 [\[dcl.fct.def.default\]](#) paragraph 3 as follows:

A function explicitly defaulted on its first declaration is implicitly inline (9.2.8 [\[dcl.inline\]](#)), and is implicitly constexpr (9.2.6 [\[dcl.constexpr\]](#)) if it ~~satisfies the requirements for a constexpr function~~ **is constexpr-suitable**.

6. Change in 11.4.7 [\[class.dtor\]](#) paragraph 9 as follows:

A defaulted destructor is a constexpr destructor if it ~~satisfies the requirements for a constexpr function~~ **is constexpr-suitable** (9.2.6 [\[dcl.constexpr\]](#)).

2642. Inconsistent use of T and C

Section: 6.5.2 [\[class.member.lookup\]](#) **Status:** ready **Submitter:** US **Date:** 2022-11-03 [P2720R0 comment US 7-035](#)

T and C are used inconsistently throughout these paragraphs.

Proposed resolution [SUPERSEDED]:

1. Change in 6.5.2 [\[class.member.lookup\]](#) paragraph 1 as follows:

A search in a scope X for a name N from a program point P is a single search in X for N from P unless X is the scope of a class or class template **T C**, in which case the following steps define the result of the search.

2. Change in 6.5.2 [\[class.member.lookup\]](#) paragraph 4 as follows:

[Note 2: If **T C** is incomplete, only base classes whose *base-specifier* appears before P are considered. If **T C** is an instantiated class, its base classes are not dependent. —end note]

3. Change in 6.5.2 [\[class.member.lookup\]](#) paragraph 6 as follows:

The result of the search is the declaration set of S(N, **T C**). If it is an invalid set, the program is ill-formed. If it differs from the result of a search in **T C** for N in a complete-class context (11.4 [\[class.mem\]](#)) of **T C**, the program is ill-formed, no diagnostic required.

4. Change in 6.5.2 [\[class.member.lookup\]](#) paragraph 7 as follows:

If N is a non-dependent *conversion-function-id*, conversion function templates that are members of **T C** are considered. For each such template F, the lookup set S(t, **T C**) is constructed, considering a function template declaration to have the name t only if it corresponds to a declaration of F (6.4.1 [\[basic.scope.scope\]](#)).

5. Change in 6.5.2 [\[class.member.lookup\]](#) paragraph 8 as follows:

[Note 4: A static member, a nested type or an enumerator defined in a base class **T B** can unambiguously be found even if an object has more than one base class subobject of type **T B**. Two base class subobjects share the non-static member subobjects of their common virtual base classes. —end note]

CWG 2022-12-02

The resolution proposed above is incorrect: T is the parameter for the overall search and C is the parameter for the S(N,C) construction. Highlight that fact in paragraph 2.

Proposed resolution (approved by CWG 2023-01-06):

1. Change in 6.5.2 [\[class.member.lookup\]](#) paragraph 1 as follows:

A search in a scope X for a name **N M** from a program point P is a single search in X for **N M** from P unless X is the scope of a class or class template T, in which case the following steps define the result of the search. [Note 1: The result differs only if **N M** is a *conversion-function-id* or if the single search would find nothing. —end note]

2. Change in 6.5.2 [\[class.member.lookup\]](#) paragraph 2 as follows:

The lookup set for **a name** N in **a class or class template** C, called S(N, C), consists of two component sets: the declaration set, a set of members named N; and the subobject set, a set of subobjects where declarations of these members were found (possibly via *using-declarations*). In the declaration set, type declarations (including injected-class-names) are replaced by the types they designate. S(N, C) is calculated as follows:

3. Change in 6.5.2 [\[class.member.lookup\]](#) paragraph 4 as follows:

... [Note 2: If **T C** is incomplete, only base classes whose *base-specifier* appears before P are considered. If **T C** is an instantiated class, its base classes are not dependent. —end note]

4. Change in 6.5.2 [\[class.member.lookup\]](#) paragraph 6 as follows:

The result of the search is the declaration set of S(**N M**, T). If it is an invalid set, the program is ill-formed. If it differs from the result of a search in T for **N M** in a complete-class context (11.4 [\[class.mem\]](#)) of T, the program is ill-formed, no diagnostic required.

5. Change in 6.5.2 [\[class.member.lookup\]](#) paragraph 7 as follows:

If **N** **M** is a non-dependent *conversion-function-id*, conversion function templates that are members of T are considered. For each such template F, the lookup set S(t, T) is constructed, considering a function template declaration to have the name t only if it corresponds to a declaration of F (6.4.1 [basic.scope.scope]). The members of the declaration set of each such lookup set, which shall not be an invalid set, are included in the result.

2658. Trivial copying of unions in core constant expressions

Section: 7.7 [expr.const] **Status:** ready **Submitter:** Hubert Tong **Date:** 2022-12-04

Consider:

```
struct A { char c; int x; };
union U { A a; };
constexpr int f() {
    U u;
    u.a.c = 1;
    u.a.x = 2;
    U v = u; // indeterminate padding bytes read!
    return u.a.x;
}
extern constexpr int x = f();
```

Subclause 7.7 [expr.const] bullet 5.11 added by P1331R2 prohibits lvalue-to-rvalue conversion of objects having indeterminate value during evaluation of a core constant expression. Trivial copy constructors of unions copy the object representation (not just the active member). The new prohibition causes cases where bytes not involved in the value presentation of the active member and having indeterminate values would prevent a union from being copied by a trivial copy constructor (for example, the padding bytes in the above case).

Note: The source of a union copy is never a prvalue within the evaluation of a trivial copy constructor because the reference parameter is bound to a glvalue.

Proposed resolution (January, 2023):

Add a new paragraph after 7.7 [expr.const] paragraph 6 as follows:

... to an object whose lifetime began within the evaluation of E.

For the purposes of determining whether *E* is a core constant expression, lvalue-to-rvalue conversion of an object of indeterminate value during the evaluation of a call to a trivial copy/move constructor or copy/move assignment operator of a union does not disqualify *E* from being a core constant expression unless the active member of the source union object contains a subobject of indeterminate value.

Proposed resolution (approved by CWG 2023-02-07):

Add a new paragraph after 7.7 [expr.const] paragraph 6 as follows:

... to an object whose lifetime began within the evaluation of E.

For the purposes of determining whether *E* is a core constant expression, the evaluation of a call to a trivial copy/move constructor or copy/move assignment operator of a union is considered to copy/move the active member of the union, if any. [Note: The copy/move of the active member is trivial. -- end note]

2659. Missing feature-test macro for lifetime extension in range-for loop

Section: 15.11 [[cpp.predefined](#)] **Status:** ready **Submitter:** CWG **Date:** 2022-12-02 [P2720R0 comment DE 038](#)

Paper P2718R0 (Wording for P2644R1 Fix for Range-based for Loop), adopted at the November, 2022 meeting, omitted a consideration of a feature-test macro, although one is desirable.

Proposed resolution (approved by CWG 2023-01-06):

Change in 15.11 [[cpp.predefined](#)] table 23 as follows:

```
__cpp_range_based_for 201603L 202211L
```

2662. Example for member access control vs. overload resolution

Section: 11.8.1 [[class.access.general](#)] **Status:** ready **Submitter:** Shafik Yaghmour **Date:** 2022-12-02

Issue 600 was resolved by P1787R6, but no example was added.

Proposed resolution (approved by CWG 2023-01-06):

Change in 11.8.1 [[class.access.general](#)] paragraph 4 as follows:

... When a *using-declarator* is named, access control is applied to it, not to the declarations that replace it. For an overload set, access control is applied only to the function selected by overload resolution.

[Example:

```
struct S {
    void f(int);
private:
    void f(double);
};

void g(S* sp) {
    sp->f(2);    // OK, access control applied after overload resolution
}
```

-- end example]

2664. Deduction failure in CTAD for alias templates

Section: 12.2.2.9 [[over.match.class.deduct](#)] **Status:** ready **Submitter:** Christof Meerwald **Date:** 2022-12-05

Subclause 12.2.2.9 [[over.match.class.deduct](#)] paragraph 3 has an exception only for deduction failure for non-deduced contexts when deducing the return type from the *defining-type-id*, but not for other cases where deduction fails according to 13.10.3.6 [[temp.deduct.type](#)] paragraph 2. For example,

```
template <class S1, class S2> struct C {
    C(...);
};

template<class T1> C(T1) -> C<T1, T1>;
template<class T1, class T2> C(T1, T2) -> C<T1 *, T2>;

template<class V1, class V2> using A = C<V1, V2>;

C c1{""};
A a1{""};
```



```
C c2{"", 1};
A a2{"", 1};
```

resulting in A having neither of these deduction guides. There is implementation divergence in the handling of this example.

Suggested resolution:

We could say that cases where P involves a template parameter and A is not of the same form (under 13.10.3.6 [temp.deduct.type] paragraph 8) are non-deduced contexts for the purpose of these deductions. That should be enough to make it clear what happens for a2, where we'd deduce $T_2 = V_2$, and not deduce anything for T_1 , but wouldn't fix a1 due to the inconsistent deductions for T_1 ; maybe this is what MSVC is doing. We could further fix a1 by allowing inconsistent deductions and treating them as if no value was deduced. Another option might be to do independent deductions for each template argument of the *simple-template-id*, and then try to merge the results for template arguments where deduction was successful; that'd be clearer that deduction can't fail, but would deduce less.

CWG 2023-02-08

In the example, A is the most trivial alias template imaginable; having this cause issues depending on the details of C is concerning.

Proposed resolution (approved by CWG 2023-02-09):

Change in 12.2.2.9 [over.match.class.deduct] paragraph 3 as follows:

... For each function or function template f in the guides of the template named by the *simple-template-id* of the *defining-type-id*, the template arguments of the return type of f are deduced from the *defining-type-id* of A according to the process in 13.10.3.6 [temp.deduct.type] with the exception that deduction does not fail if not all template arguments are deduced. **If deduction fails for another reason, proceed with an empty set of deduced template arguments** Let g denote the result of substituting these deductions into f. ...

2667. Named module imports do not import macros

Section: 15.5 [cpp.import] **Status:** ready **Submitter:** Richard Smith **Date:** 2022-12-16

It should be clarified via an example or a note that named module imports do not make macros available.

Proposed resolution (approved by CWG 2023-01-06):

1. Change in 10.3 [module.import] paragraph 7 as follows:

... These rules can in turn lead to the importation of yet more translation units. **[Note: Such indirect importation does not make macros available, because a translation unit is a sequence of tokens in translation phase 7 (5.2 [lex.phases]). Macros can be made available by directly importing header units as described in 15.5 [cpp.import]. -- end note]**

2. Add to the example in 15.5 [cpp.import] paragraph 8 as follows:

```
import "a.h";    // point of definition of #1, #2, and #3, point of undefinition of #1 in "e.h"
import "d.h";    // point of definition of #4 and #5 in "e.h"
int a = Y;       // OK, active macro definitions #2 and #4 are valid redefinitions
int c = Z;       // error: active macro definitions #3 and #5 are not valid redefinitions of Z
```

Module unit f:

```
export module f;
export import "a.h";

int a = Y;    // OK
```

Translation unit #1:

```
import f;
int x = Y;    // error: Y is neither a defined macro nor a declared name
```

-- end example]

2673. User-declared spaceship vs. built-in operators

Section: 12.2.2.3 [[over.match.oper](#)] **Status:** ready **Submitter:** Barry Revzin **Date:** 2022-12-30

Consider:

```
#include <compare>

enum class E : int {
    Lo = 0,
    Hi = 1
};

constexpr auto operator<=>(E lhs, E rhs) -> std::strong_ordering {
    return (int)rhs <=> (int)lhs;
}

// everybody agrees this is true
static_assert((E::Lo <=> E::Hi) == std::strong_ordering::greater);

// gcc rejects this, msvc and clang accept
static_assert(E::Lo > E::Hi); // #1
```

The intent here is for the user-provided operator<=> to suppress the built-in operator<=> for E. And gcc, clang, and msvc all agree that this does happen when the comparison expression explicitly uses a <=> b.

But when the comparison expression is a @ b for one of the relational operators, gcc disagrees, conforming to 12.2.2.3 [[over.match.oper](#)] bullet 3.3:

For all other operators, the built-in candidates include all of the candidate operator functions defined in 12.5 [[over.built](#)] that, compared to the given operator, ... do not have the same parameter-type-list as any non-member candidate that is not a function template specialization.

The issue is that, for #1, the user-provided operator<=> is not a non-member candidate, but a rewritten candidate. A similar situation arises for a user-declared operator==, which will be called for e1 == e2, but not for e1 != e2. Again, clang and MSVC disagree.

Proposed resolution (January, 2023) [SUPERSEDED]:

Change 12.2.2.3 [[over.match.oper](#)] bullet 3.3.4 as follows:

- ...
- do not have the same parameter-type-list as any non-member **or rewritten** candidate that is not a function template specialization.

Proposed resolution (approved by CWG 2023-02-10):

Change 12.2.2.3 [[over.match.oper](#)] bullet 3.3.4 as follows:

- ...
 - do not have the same parameter-type-list as any non-member candidate **or rewritten non-member candidate** that is not a function template specialization.
-

2674. Prohibit explicit object parameters for constructors

Section: 11.4.5.1 [[class.ctor.general](#)] **Status:** ready **Submitter:** Erich Keane **Date:** 2022-01-11

Constructors are not intended to have explicit object parameters, but the standard is missing a corresponding prohibition.

Proposed resolution (approved by CWG 2023-01-06):

Add a new paragraph after 11.4.5.1 [[class.ctor.general](#)] paragraph 7 as follows:

A constructor shall not be a coroutine.

A constructor shall not have an explicit object parameter (9.3.4.6 [[dcl.fct](#)]).

2678. `std::source_location::current` is unimplementable

Section: 6.3 [[basic.def.odr](#)] **Status:** ready **Submitter:** Richard Smith **Date:** 2023-01-07

Consider:

```
#include <source_location>

inline char *f() {
    static char array[std::source_location::current().line()];
    return array;
}
```

The sequence of tokens comprising the definition of `f` can appear in multiple translation units, on different lines. The one-definition rule is not violated. Thus, there is a single function `f` in the program with a unique static local object array, but that object would have a different type in each translation unit. It is unclear how to implement this, absent the conservative approach of always returning a value-initialized object from `std::source_location::current`, which would defeat its purpose.

Possible approaches to resolve this issue might include:

- Do not mark the `source_location` accessors `constexpr`; they are not deterministic.
- Prohibit calls to `source_location` accessors during constant evaluation unless the `current()` call happened in a context that can only appear once in a program.
- Do not mark the `current` function `constexpr` / `constexpr`.
- Change the ODR to prohibit calling the `current` function in an inline function or in a function template.

2023-01-08

Forwarded to LWG / LEWG via [cplusplus/papers#1416](#), by decision of the CWG chair.

EWG 2023-02-07

EWG approves of the approach for CWG2678 of changing the ODR to make use of `source_location` in a way that causes an inline function/function template/etc to 'be different' be an ODR violation.

Proposed resolution (approved by CWG 2023-02-08):

Add a new bullet after 6.3 [[basic.def.odr](#)] bullet 14.8 as follows:

- In each such definition, `const` objects with static or thread storage duration shall be constant-initialized if the object is constant-initialized in any such definition.
- In each such definition, corresponding manifestly constant-evaluated expressions that are not value-dependent shall have the same value (7.7 [[expr.const](#)], 13.8.3.4 [[temp.dep.constexpr](#)]).

- In each such definition, the overloaded operators referred to, the implicit calls to conversion functions, constructors, operator new functions and operator delete functions, shall refer to the same function.

2681. Deducing member array type from string literal

Section: 12.2.2.9 [[over.match.class.deduct](#)] **Status:** ready **Submitter:** Jonathan Caves **Date:** 2020-03-17

Consider:

```
template<typename T, std::size_t N>
struct A {
    T array[N];
};

A a = { "meow" };
```

The current wording says in 12.2.2.9 [[over.match.class.deduct](#)] bullet 1.8:

- if e_i is of array type and x_i is a *braced-init-list* or *string-literal*, T_i is an rvalue reference to the declared type of e_i , and
- ...

This will fail overload resolution, because a string literal (which is an lvalue) does not match a parameter of type $T \ (\&\&)[N]$.

Proposed resolution (approved by CWG 2023-02-07):

1. Change in 12.2.2.9 [[over.match.class.deduct](#)] paragraph 1.8 as follows:

- if e_i is of array type and x_i is a *braced-init-list* ~~or *string-literal*~~, T_i is an rvalue reference to the declared type of e_i , and
- **if e_i is of array type and x_i is a *string-literal*, T_i is an lvalue reference to the const-qualified declared type of e_i , and**
- ...

2. Append to the example in 12.2.2.9 [[over.match.class.deduct](#)] paragraph 2 as follows:

```
G g(true, 'a', 1); // OK, deduces G<char, bool>
```

```
template<class T, std::size_t N>
struct H {
    T array[N];
};
template<class T, std::size_t N>
struct I {
    volatile T array[N];
};
template<std::size_t N>
struct J {
    unsigned char array[N];
};

H h = { "abc" }; // OK, deduces H<char, 4> (not T = const char)
I i = { "def" }; // OK, deduces I<char, 4>
J j = { "ghi" }; // error: cannot bind reference to array of unsigned char to array of char in deduction
```

2682. Templated function vs. function template

Section: 13.1 [temp.pre] **Status:** ready **Submitter:** Matthew House **Date:** 2023-01-11

In 13.1 [temp.pre] paragraph 8, the phrase "templated entity" is defined. The derived term "templated function" is never actually defined, but is intended to apply to function templates as well as non-template members of class templates. Similarly, the phrases "templated variable" and "templated class" should be properly defined.

Proposed resolution (approved by CWG 2023-01-27):

1. Change in 9.2.9.6.1 [dcl.spec.auto.general] paragraph 12 as follows:

Return type deduction for a templated ~~entity that is a function or function template~~ with a placeholder in its declared type occurs when the definition is instantiated even if the function body contains a return statement with a non-type-dependent operand.

2. Change in 13.1 [temp.pre] paragraph 8 as follows:

[Note 6: A local class, a local or block variable, or a friend function defined in a templated entity is a templated entity. —end note]

A templated function is a function template or a function that is templated. A templated class is a class template or a class that is templated. A templated variable is a variable template or a variable that is templated.

2685. Aggregate CTAD, string, and brace elision

Section: 12.2.2.9 [over.match.class.deduct] **Status:** ready **Submitter:** Jason Merrill **Date:** 2020-07-14

Consider:

```
template <class T>
struct A {
    T ar[4];
};
A a = { "foo" };
```

Subclause 12.2.2.9 [over.match.class.deduct] bullet 1.5 specifies:

For each x_i , let e_i be the corresponding aggregate element of C or of one of its (possibly recursive) subaggregates that would be initialized by x_i (9.4.2 [dcl.init.aggr]) if

- brace elision is not considered for any aggregate element that has a dependent non-array type or an array type with a value-dependent bound, and
- ...

The normative rule does not properly consider arrays with dependent element type, initialized by a string literal. MSVC accepts, gcc and clang reject the example.

Suggested resolution [SUPERSEDED]:

Change in 12.2.2.9 [over.match.class.deduct] bullet 1.5 as follows:

For each x_i , let e_i be the corresponding aggregate element of C or of one of its (possibly recursive) subaggregates that would be initialized by x_i (9.4.2 [dcl.init.aggr]) if

- brace elision is not considered for any aggregate element that has
 - a dependent non-array type ~~or~~,
 - an array type with a value-dependent bound, **or**
 - **a string literal as the corresponding initializer and that has an array type whose element type is a (possibly cv-qualified) template parameter or is a dependent type specified with a *qualified-id* whose *nested-name-specifier* is dependent or with a *decltype-specifier*; and**

- ...

Proposed resolution (approved by CWG 2023-02-09):

Change in 12.2.2.9 [[over.match.class.deduct](#)] bullet 1.5 as follows:

For each x_i , let e_i be the corresponding aggregate element of C or of one of its (possibly recursive) subaggregates that would be initialized by x_i (9.4.2 [[dcl.init.aggr](#)]) if

- brace elision is not considered for any aggregate element that has
 - a dependent non-array type ~~or~~,
 - an array type with a value-dependent bound, **or**
 - **an array type with a dependent array element type and x_i is a string literal;** and
- ...

2690. Semantics of defaulted move assignment operator for unions

Section: 11.4.6 [[class.copy.assign](#)] **Status:** ready **Submitter:** Cassio Neri **Date:** 2023-01-20

Subclause 11.4.6 [[class.copy.assign](#)] paragraph 13 does not specify the semantics of a defaulted move assignment operator:

The implicitly-defined copy assignment operator for a union x copies the object representation (6.8.1 [[basic.types.general](#)]) of x

In contrast, the corresponding rule for move constructors is present in 11.4.5.3 [[class.copy.ctor](#)] paragraph 15:

The implicitly-defined copy/move constructor for a union x copies the object representation (6.8.1 [[basic.types.general](#)]) of x

Proposed resolution (approved by CWG 2023-01-27):

Change in 11.4.6 [[class.copy.assign](#)] paragraph 13 as follows:

The implicitly-defined copy/**move** assignment operator for a union x copies the object representation (6.8.1 [[basic.types.general](#)]) of x

2691. *hexadecimal-escape-sequence* is too greedy

Section: 5.13.3 [[lex.ccon](#)] **Status:** ready **Submitter:** Fraser Gordon **Date:** 2023-01-26

The lexer grammar production *hexadecimal-escape-sequence* matches text of the form `\x{20}ab` due to its recursive definition.

Proposed resolution (approved by CWG 2023-01-27):

Change in 5.13.3 [[lex.ccon](#)] as follows:

```
hexadecimal-escape-sequence :
  \x hexadecimal-digit simple-hexadecimal-digit-sequence
hexadecimal-escape-sequence hexadecimal-digit
  \x{ simple-hexadecimal-digit-sequence }
```

2695. Semantic ignorability of attributes

Section: 9.12.1 [[dcl.attr.grammar](#)] **Status:** ready **Submitter:** Timur Doumler **Date:** 2023-02-09

EWG resolved to reflect the understanding of semantic ignorability of attributes in a note.

Proposed resolution (approved by CWG 2023-02-09):

Add to 9.12.1 [[dcl.attr.grammar](#)] paragraph 6 as follows:

[*Note 4*: A program is ill-formed if it contains an attribute specified in 9.12 [[dcl.attr](#)] that violates the rules specifying to which entity or statement the attribute can apply or the syntax rules for the attribute's *attribute-argument-clause*, if any. —*end note*] [***Note*: The *attributes* specified in 9.12 [[dcl.attr](#)] have optional semantics: given a well-formed program, removing all instances of any one of those *attributes* results in a program whose set of possible executions (4.1.2 [[intro.abstract](#)]) for a given input is a subset of those of the original program for the same input, absent implementation-defined guarantees with respect to that *attribute*. -- end note**]